

UEMF Présence

Système de gestion des présences par QR code rotatif

Modélisation d'un problème universitaire en optimisation combinatoire, puis transformation en application web complète, sécurisée, déployée en production.

Réalisé par	Mouhssine El Boumshouli
Encadrant	Pr Ahmed El Hilali Alaoui (Directeur académique EIDIA)
Formation	EIDIA — UEMF, Semestre 5
Année universitaire	2025–2026
Stack	Next.js 16 · React 19 · TypeScript · Prisma 6 · PostgreSQL (Neon)
Déploiement	Vercel — HTTPS automatique
Code source	github.com/MouhssineElBoumshouli/Student-Attendance-System

Table des matières

§ 1	Le problème à résoudre
§ 2	Recherche Opérationnelle — rappels
§ 3	Notre solution — vue d'ensemble
§ 4	Formulation en Recherche Opérationnelle
§ 5	Fonction objectif multi-critère
§ 6	Les cinq contraintes du modèle
§ 7	Le mécanisme QR rotatif
§ 8	Quatre couches anti-fraude
§ 9	Stack technique
§ 10	Architecture et flux de données
§ 11	Modèle de données
§ 12	Sécurité API par rôle
§ 13	Cycle de vie d'une séance
§ 14	Bilan et limites assumées
§ 15	Glossaire technique

§ 1 — Le problème à résoudre

À l'Université Euro-Méditerranéenne de Fès, la présence en cours est saisie manuellement via la plateforme Konosys : le professeur appelle chaque étudiant par son nom, attend la réponse, puis coche la liste. Pour une promotion de 30 à 60 étudiants, cela coûte 5 à 15 minutes de temps de cours, à **chaque séance**. Et au-delà du temps perdu, le procédé est fragile : erreurs, oublis, et surtout *aucun* mécanisme contre la fraude — un voisin peut répondre « présent » pour un absent sans que personne ne le remarque.

DÉFINITION — PROBLÈME D'OPTIMISATION

En Recherche Opérationnelle, un **problème d'optimisation** consiste à trouver, parmi toutes les solutions possibles, celle qui maximise (ou minimise) une *quantité d'intérêt* appelée **fonction objectif**, sous un ensemble de **contraintes** à respecter. Ici, la quantité à maximiser est la fiabilité du système de présence ; les contraintes sont l'identité de l'étudiant, sa présence physique en salle, et l'intégrité du token QR.

1.1 Les quatre pathologies du système actuel

Pathologie	Description	Impact
Temps perdu	Appel un par un pour 30 à 60 étudiants au début du cours.	5 à 15 min / séance
Erreurs humaines	Homonymes, oublis, prénoms mal entendus, coches décalées.	2 à 5 %
Fraude triviale	Un étudiant répond « présent » pour un absent.	0 contrôle
Données figées	Aucune agrégation, aucune analyse statistique exploitable.	Manuel uniquement

Tableau 1 — Pathologies de la prise de présence manuelle

Ces quatre pathologies ne sont pas indépendantes : elles se nourrissent les unes les autres. Le temps perdu pousse le professeur à expédier l'appel, ce qui augmente les erreurs et la fraude, et l'absence de données empêche de mesurer le problème pour le corriger. C'est le profil classique d'un **problème de processus mal posé** — donc une cible naturelle pour une approche RO.

1.2 Comparatif ancien processus vs. notre système

Critère	Konosys (actuel)	UEMF Présence (QR)
Temps de prise	5–15 min / séance	15–30 secondes
Méthode	Appel vocal manuel	Scan QR rotatif (smartphone)
Anti-fraude	Aucun	4 couches indépendantes
Mise à jour côté prof	Différée (fin de cours)	~5 sec après le scan
Export rapports	Manuel	CSV à un clic (RFC 4180)

Critère	Konosys (actuel)	UEMF Présence (QR)
Taux d'erreur de saisie	2–5 %	~0 % (automatisé)
Coût annuel	Licence Konosys	0 € (Vercel + Neon gratuits)

Tableau 2 — Comparatif des deux processus

§ 2 — Recherche Opérationnelle : rappels

Avant d'attaquer notre modèle, un mot sur la discipline elle-même. La Recherche Opérationnelle (RO) est l'application des méthodes scientifiques — mathématiques, statistiques, modélisation — à la *prise de décision* dans des organisations complexes. Née pendant la Seconde Guerre mondiale (logistique militaire), elle équipe aujourd'hui les compagnies aériennes (planification d'équipages), les hôpitaux (affectation de blocs opératoires), les transporteurs (tournées de livraison), etc.

2.1 Les trois ingrédients d'un modèle RO

- **Variables de décision** — les inconnues que l'on cherche à fixer. Ce sont les leviers sur lesquels on agit.
- **Fonction objectif** — la quantité scalaire qui mesure la qualité d'une solution. On cherche à la maximiser ou à la minimiser.
- **Contraintes** — les conditions que toute solution acceptable doit satisfaire.

DÉFINITION — OPTIMISATION COMBINATOIRE

Famille de problèmes RO où les variables de décision prennent leurs valeurs dans un ensemble *fini et discret* (souvent $\{0, 1\}$). Exemples classiques : le voyageur de commerce, l'affectation, le sac à dos. Par opposition à l'*optimisation continue* (programmation linéaire, non linéaire), où les variables prennent des valeurs réelles.

2.2 Pourquoi notre problème est combinatoire

Pour chaque couple (étudiant, séance), une seule question binaire : *cet étudiant était-il présent à cette séance ?* La réponse est dans $\{0, 1\}$. Avec 201 étudiants et, disons, 100 séances par semestre, cela fait **20 100 variables binaires** à fixer — chacune contrainte par les règles d'identité (la personne qui scanne est bien l'étudiant) et de présence physique (il était bien en salle).

POURQUOI MULTI-OBJECTIF ?

Réduire le temps d'appel est *un* objectif ; vérifier que chaque scan est authentique en est *un autre* ; et garder un taux de couverture élevé (peu d'étudiants oubliés) en est *un troisième*. Ces trois objectifs sont en partie en concurrence : ajouter des vérifications (GPS, fingerprint) ralentit le scan ; supprimer les vérifications accélère mais ouvre la porte à la fraude. La RO multi-critère est l'outil approprié pour *pondérer* ces compromis.

2.3 La méthode de scalarisation pondérée

Quand on a plusieurs objectifs Z_1, Z_2, Z_3 , on les combine en un objectif unique Z par une **somme pondérée convexe** :

$$Z = \alpha \cdot Z_1 + \beta \cdot Z_2 + \gamma \cdot Z_3, \quad \text{avec } \alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0$$

Les coefficients α, β, γ représentent l'*importance relative* que le décideur accorde à chaque objectif. Faire varier ces poids et observer comment la solution optimale change, c'est le cœur de l'**analyse de sensibilité** — un classique en RO.

§ 3 — Notre solution en bref

Une application web déployée sur Vercel, accessible depuis n'importe quel navigateur — ordinateur du professeur pour afficher le QR en classe, smartphone de l'étudiant pour scanner. Trois rôles distincts, chacun avec son tableau de bord et ses permissions, vérifiées *côté serveur* à chaque requête.

3.1 Les trois rôles

Rôle	Capacités
Administrateur	Gestion complète : départements, filières, groupes, salles (avec coordonnées GPS), professeurs, étudiants, cours, analytiques globales.
Professeur	Voir ses cours, planifier des séances, activer une séance (génère le QR rotatif), suivre les scans en temps quasi-réel, corriger manuellement, exporter le rapport CSV.
Étudiant	Scanner le QR affiché en classe via la caméra, consulter son historique et son taux de présence par cours.

Tableau 3 — Capacités par rôle

3.2 Le flux principal en sept étapes

1. Le **professeur** ouvre son tableau de bord et clique « Activer » sur la séance prévue.
2. Le **serveur** tire un secret cryptographique aléatoire de 32 octets, le stocke en base, et pré-crée une ligne *ABSENT* pour chaque étudiant des groupes du cours.
3. Le **professeur** affiche la page QR plein écran. Le QR change toutes les 10 secondes, calculé à partir du secret.
4. Chaque **étudiant** ouvre la page « Scanner » sur son téléphone, autorise la caméra et le GPS.
5. Le téléphone **décodes le QR** et envoie au serveur le token, la position GPS, et une empreinte de l'appareil.
6. Le **serveur** vérifie l'authentification, la validité du token (HMAC), la distance à la salle (Haversine), l'unicité de l'appareil, puis bascule la ligne *ABSENT* en *PRESENT* (ou *LATE* si > 15 min).
7. Le **tableau de bord du professeur** se met à jour automatiquement (~5 sec) et affiche le nouveau présent.

EN PRATIQUE — LA PROMOTION 2025-2026

201 étudiants EIDIA répartis en 8 groupes (4 en Génie Informatique, 4 en IA et Science des Données), **4 professeurs** dont le Pr Ahmed El Hilali Alaoui en Recherche Opérationnelle, **4 cours** et **4 salles** géolocalisées sur le campus UEMF de Fès. Toutes ces données sont chargées dans la base de production via un script de seed reproductible (prisma/seed.ts).

§ 4 — Formulation en Recherche Opérationnelle

On modélise le problème comme une **optimisation combinatoire à variables binaires sous contraintes**. La formulation suit la structure standard RO en trois étapes : (1) définir les ensembles, (2) définir les variables de décision, (3) définir l'objectif et les contraintes (§5 et §6).

4.1 Les ensembles

Un *ensemble*, en RO, est simplement une liste finie d'éléments sur lesquels on va indiquer nos variables.

```
-- Ensembles fondamentaux --
S = { s■, s■, ..., s■ } // séances planifiées (cours)
E = { e■, e■, ..., e■ } // étudiants inscrits
T = { t■, t■, ..., t■ } // fenêtres de rotation du QR ( $\Delta = 10$  s)
R = { r■, r■, ..., r■ } // salles, avec (lat, lon, rayon)
G = { g■, g■, ..., g■ } // groupes (par filière, semestre)

-- Application à notre cas concret --
|E| = 201 // étudiants EIDIA S5
|G| = 8 // 4 GI + 4 IASD
|R| = 4 // Amphi A, Amphi B, Salle 204, Labo Info 1
 $\Delta = 10$  s // fenêtre de validité d'un QR
```

4.2 Les variables de décision

Une **variable de décision** représente une grandeur que le système va calculer. Dans notre modèle, elles sont presque toutes *binaires* ($\{0, 1\}$) — c'est ce qui rend le problème combinatoire.

Variable	Domaine	Signification
x_{ij}	$\{0, 1\}$	= 1 si l'étudiant e_i est marqué présent à la séance s_j
$\blacksquare_{ij}$	$\{0, 1\}$	= 1 si e_i est arrivé en retard à s_j ($\Delta t > 15$ min)
v_{ij}	$\{0, 1\}$	= 1 si la présence est vérifiée (GPS dans le rayon ET appareil unique)
q_{jk}	hex (8)	token HMAC affiché par s_j pendant la fenêtre t_k
d_{ij}	R^+	distance GPS (m) entre e_i et la salle de s_j — Haversine
Δt_{ij}	R^+	délai d'arrivée de e_i après le début de s_j (min)
f_{ij}	SHA-256	empreinte de l'appareil utilisé par e_i pour scanner s_j

Tableau 4 — Variables de décision du modèle

POURQUOI DES VARIABLES BINAIRES PLUTÔT QUE CONTINUES ?

Un étudiant est *soit* présent *soit* absent — il n'existe pas de présence « à 73 % ». De même, un token est valide ou ne l'est pas, un appareil est dans le rayon ou ne l'est pas. La nature discrète de la décision se reflète dans la nature binaire des variables. La conséquence est que le problème est combinatoire et non continu.

§ 5 — Fonction objectif multi-critère

Notre système poursuit trois objectifs simultanés. On les combine via une somme pondérée convexe pour obtenir un critère unique Z à maximiser. C'est la **méthode de scalarisation**, l'approche la plus classique pour transformer un problème multi-objectif en problème mono-objectif équivalent.

5.1 Les trois sous-objectifs

- **Z_{C} — Couverture automatique.** La part d'étudiants dont la présence est saisie sans aucune intervention du professeur.
- **Z_{V} — Vérification.** La part des présences automatiques qui passent *toutes* les vérifications (GPS dans le rayon + appareil unique).
- **Z_{T} — Efficacité temporelle.** Le gain de temps par rapport à l'appel manuel.

5.2 Forme analytique de Z

Maximiser $Z = \alpha \cdot Z_{\text{C}} + \beta \cdot Z_{\text{V}} + \gamma \cdot Z_{\text{T}}$, $\alpha + \beta + \gamma = 1$, ($\alpha=0,4$, $\beta=0,3$, $\gamma=0,3$ par défaut)

```
--  $Z_{\text{C}}$  : taux de couverture automatique --  
 $Z_{\text{C}} = (1 / |S|) \cdot \sum [ (\sum x_{ij}) / |E_i| ]$   
    pour chaque séance, on calcule la fraction d'étudiants présents,  
    puis on moyenne sur l'ensemble des séances  
  
--  $Z_{\text{V}}$  : taux de présences vérifiées (GPS + appareil unique) --  
 $Z_{\text{V}} = (\sum v_{ij}) / \max(1, \sum x_{ij})$   
    part des scans présents qui ont passé toutes les vérifications  
  
--  $Z_{\text{T}}$  : efficacité temporelle --  
 $Z_{\text{T}} = 1 - (T_{\text{QR}} / T_{\text{manuel}})$   
    avec  $T_{\text{QR}} \approx 25$  s et  $T_{\text{manuel}} \approx 600$  s  $\rightarrow Z_{\text{T}} \approx 0,958$ 
```

EXEMPLE CHIFFRÉ SUR UNE SÉANCE TYPE

Sur un cours de 50 étudiants, supposons que 48 scannent avec succès et qu'aucun ne soit retardataire : $Z_{\text{C}} = 48/50 = 0,96$. Si 45 de ces 48 ont aussi un GPS valide et un appareil unique : $Z_{\text{V}} = 45/48 \approx 0,938$. Le temps total a été d'environ 25 s contre 8 min en appel manuel : $Z_{\text{T}} = 1 - 25/480 \approx 0,948$. Avec $\alpha=0,4$, $\beta=0,3$, $\gamma=0,3$: $Z \approx 0,950$. Soit une fiabilité globale estimée de 95,0 %.

5.3 Analyse de sensibilité aux poids

α	β	γ	Priorité	Z estimé
0,40	0,30	0,30	Équilibrée (défaut)	0,937
0,60	0,20	0,20	Privilégie la couverture	0,943
0,20	0,60	0,20	Privilégie la vérification	0,931
0,20	0,20	0,60	Privilégie la rapidité	0,945

Tableau 5 — Sensibilité de Z aux poids ($Z_{\text{■}}=0,95$; $Z_{\text{■}}=0,90$; $Z_{\text{■}}=0,958$)

Les valeurs $Z_{\text{■}}$, $Z_{\text{■}}$, $Z_{\text{■}}$ utilisées ci-dessus sont des *estimations cibles*. Elles seront recalibrées à partir des données réelles après une période d'usage en production.

§ 6 — Les cinq contraintes du modèle

Chaque contrainte mathématique du modèle est traduite en mécanisme *vérifiable* côté serveur — c'est le pont direct entre la formalisation RO et le code. Si une contrainte n'est pas respectée, le scan est refusé ou marqué non-vérifié.

DÉFINITION — CONTRAINTE

Condition que toute solution acceptable doit satisfaire. Une contrainte peut être : (a) une *égalité*, (b) une *inégalité*, ou (c) une *contrainte logique* reliant plusieurs variables. La région de l'espace des décisions qui satisfait toutes les contraintes est dite **réalisable**.

6.1 C1 — Unicité de la présence

Un étudiant ne peut être marqué qu'une seule fois par séance. Si le même étudiant scanne deux fois, la seconde tentative est rejetée par la base de données *avant* même d'atteindre le code applicatif.

$$x_{ij} + \blacksquare_{ij} \leq 1, \quad \forall i \in E, \forall j \in S$$

Implémentation : contrainte SQL UNIQUE(sessionId, studentId) sur la table attendances. PostgreSQL refuse l'insertion en violation de cette contrainte avec le code d'erreur P2002.

6.2 C2 — Contraintes temporelles

Un token QR n'est cryptographiquement valide que pendant sa fenêtre de 10 secondes. Pour gérer les scans à cheval sur la limite, on accepte aussi la fenêtre *précédente* (tolérance d'un cran). Un scan arrivant plus de 15 minutes après le début programmé bascule l'étudiant en statut *EN RETARD*.

$$q_{jk} \text{ valide } \blacksquare \blacksquare t_{\text{scan}} / \Delta \blacksquare \in \{ k_k, k_k - 1 \} \\ \Delta t_{ij} > 15 \text{ min } \blacksquare \blacksquare_{ij} = 1$$

Implémentation : le serveur recalcule le HMAC pour la fenêtre courante et la précédente, et compare en temps constant via `crypto.timingSafeEqual`. La tolérance d'horloge client-serveur est plafonnée à ± 60 secondes.

6.3 C3 — Contrainte géographique (Haversine)

L'étudiant doit se trouver à l'intérieur d'un rayon configurable autour des coordonnées GPS de la salle. La distance est calculée par la **formule de Haversine**, qui donne la longueur du plus court arc géodésique sur une sphère.

$$a = \sin^2((\phi_{\blacksquare} - \phi_{\blacksquare}) / 2) + \cos(\phi_{\blacksquare}) \cdot \cos(\phi_{\blacksquare}) \cdot \sin^2((\lambda_{\blacksquare} - \lambda_{\blacksquare}) / 2) \\ d_{ij} = 2R \cdot \arcsin(\sqrt{a}), \quad R = 6\,371 \text{ km} \\ v_{ij} = 1 \blacksquare d_{ij} \leq R_{\text{salle}} \wedge (\text{autres conditions de C5})$$

Pourquoi Haversine et pas Pythagore ? Pythagore suppose un plan euclidien — faux à l'échelle de la Terre. À l'échelle d'un campus, l'erreur serait négligeable, mais Haversine est tout aussi simple à coder et correct par construction.

Rayons configurés par salle : Amphi A & B = 150 m (grands amphis, GPS moins précis en intérieur), Salle 204 = 80 m, Labo Info = 60 m (petite pièce).

6.4 C4 — Contrainte cryptographique (HMAC-SHA256)

Les tokens sont générés par **HMAC-SHA256**, schéma inspiré du protocole TOTP (RFC 6238 — celui qui équipe Google Authenticator). La clé secrète $secret_j$ est tirée aléatoirement (32 octets via `crypto.randomBytes`) au moment de l'activation de la séance. Elle reste en base, jamais transmise au navigateur.

$$q_{jk} = \text{HMAC-SHA256}(secret_j, k)[0..7] , \quad (8 \text{ hex} \Rightarrow 32 \text{ bits utiles})$$
$$P(\text{deviner } q_{jk}) = 1 / 2^{32} \approx 2,3 \times 10^{-10}$$

Sécurité : sans le secret, deviner le token sortant revient à un brute-force sur l'espace de sortie (~4,3 milliards d'essais). Même à 10 000 requêtes/seconde (déjà bloqué côté serveur), il faudrait des jours pour deviner un seul token — qui aurait expiré 10 s plus tard de toute façon.

6.5 C5 — Empreinte d'appareil unique

Un même appareil physique ne peut pas valider deux étudiants différents pour la même séance. La détection se fait par **empreinte SHA-256 de (UserAgent + résolution écran + langue)**. Si deux scans présentent la même empreinte dans la même séance, les *deux* présences passent en « non vérifiée » — le professeur tranche manuellement après examen.

$$f_{ij} = f_{i'j} \wedge i \neq i' \quad \blacksquare \quad v_{ij} = 0 \wedge v_{i'j} = 0$$

LIMITE ASSUMÉE

Deux smartphones de même modèle, même OS, même langue produiront la *même empreinte*. C'est donc un mécanisme heuristique qui décourage la fraude opportuniste, pas une preuve forte. C'est précisément *pour cette raison* que la conséquence est « non vérifiée » (à examiner) et non « refusée » : on ne veut pas pénaliser deux frères qui ont le même téléphone.

§ 7 — Le mécanisme QR rotatif

C'est la pièce centrale du projet, qui rend la fraude par photo impossible : **le QR change toutes les 10 secondes**. Toute capture d'écran est obsolète avant d'avoir pu être partagée. Le défi technique a été double : (1) construire un schéma cryptographique correct pour générer ces tokens, et (2) obtenir un affichage *visuellement stable* pendant chaque fenêtre de 10 s — pas de clignotement.

DÉFINITION — QR CODE

Un *Quick Response code* est un code-barres 2D, capable d'encoder du texte ASCII jusqu'à environ 4 296 caractères. On peut y embarquer une URL, un JSON, une chaîne signée. Notre QR encode un JSON minimal : { "s": sessionId, "t": token, "ts": windowStart }.

7.1 Qu'est-ce que HMAC-SHA256 ?

HMAC = *Hash-based Message Authentication Code*. C'est un mécanisme cryptographique standardisé qui prend deux entrées — une *clé secrète* et un *message* — et produit une *signature* qui prouve simultanément :

- L'**intégrité** du message : il n'a pas été modifié.
- L'**authenticité** : seul celui qui possède la clé secrète aurait pu produire cette signature.

SHA-256 est la fonction de hachage interne utilisée — elle produit 256 bits de sortie. HMAC-SHA256 est utilisé partout : AWS, Stripe, GitHub webhooks, JWT, etc.

7.2 Comment le token est généré

```
// À l'activation de la séance, on tire un secret aléatoire de 32 octets
secret■ = randomBytes(32)           // stocké en base, jamais envoyé

// À chaque requête /api/sessions/{id}/qr-token
now      = floor(Date.now() / 1000) // timestamp courant en s
k        = floor(now / 10)           // indice de la fenêtre courante (Δ = 10 s)
windowStart = k × 10                 // début de la fenêtre en epoch
q■■      = HMAC-SHA256(secret■, k)[0..7] // 8 premiers hex
expiresAt  = (k + 1) × 10 × 1000     // fin de fenêtre en ms

// Le payload du QR est stable pendant toute la fenêtre
payload = JSON.stringify({ s: sessionId, t: q■■, ts: windowStart })
```

7.3 Pourquoi windowStart et non now ?

Dans la première version, le payload incluait `ts = now`. Conséquence : chaque sondage du serveur renvoyait un payload *différent*, même si le token `t` était identique. Le canvas QR se redessinait à chaque réponse : l'image clignotait visiblement.

LE BUG CORRIGÉ

Symptôme : clignotement du QR toutes les 2 secondes, désorientant pour les étudiants en train de scanner. **Cause racine** : le payload du QR contenait l'horodatage *de la requête*, qui change à chaque appel. **Correctif** : stocker à la place `windowStart = k × Δ`, identique pendant toute la fenêtre. Le payload — donc le QR — devient stable pendant 10 secondes, puis bascule d'un coup à la fenêtre suivante.

7.4 Polling intelligent — un seul fetch par fenêtre

Sonder à intervalle fixe est gaspilleur de bande passante et provoque des micro-décalages d'affichage. Notre approche : planifier le prochain fetch exactement **200 ms après l'expiration** de la fenêtre courante. Le compte à rebours à l'écran est dérivé continûment de `expiresAt - Date.now()` et rafraîchi toutes les 200 ms — visuellement fluide.

7.5 Comment l'étudiant scanne et soumet

Smartphone étudiant	API /attendance	PostgreSQL
caméra → jsQR		
décode payload		
+ GPS (<code>navigator.geolocation</code>)		
+ fingerprint		
POST { token, ts, lat, lon, deviceInfo }		
----->		
	1. requireApiRole(STUDENT)	
	2. studentId ← session JWT ← pas du body !	
	3. recalc HMAC(secret, k)	
	timingSafeEqual	
	4. Haversine(stu, salle)	
	5. SHA-256(deviceInfo)	
	6. statut = retard ?	
	---- UPDATE attendance ---->	
<- { status, verified, distance }		

POURQUOI studentId VIENT DU JWT, PAS DU BODY ?

Sans cette protection, un étudiant malin pourrait ouvrir la console de son navigateur et envoyer une requête POST en spécifiant l'ID d'un absent. Notre serveur ignore complètement le `studentId` envoyé dans le corps de la requête — il le lit depuis le token JWT signé qui prouve l'identité de l'utilisateur connecté. C'est la **seule** garantie qu'un scan ne peut marquer présent que la personne réellement connectée.

§ 8 — Quatre couches anti-fraude

Les contraintes C2 à C5 du modèle se traduisent en quatre couches de défense techniques, *indépendantes* les unes des autres. C'est le principe de **défense en profondeur** : même si une couche tombe (par exemple, l'étudiant a refusé la géolocalisation), les autres restent actives et limitent l'impact.

DÉFINITION — DÉFENSE EN PROFONDEUR

Stratégie de sécurité héritée du domaine militaire : plutôt qu'un mur unique, on superpose plusieurs lignes de défense de natures différentes. Un attaquant qui contourne l'une est arrêté par la suivante.

#	Couche	Bloque
1	Rotation des tokens (10 s) — nouveau token HMAC-SHA256 toutes les 10 secondes. Le token change avant qu'une photo ne puisse circuler.	capture d'écran partagée
2	Géofence GPS (Haversine) — distance au centre de la salle calculée à la réception. Hors rayon → présence marquée non vérifiée.	scan à distance, visio
3	Empreinte d'appareil — SHA-256 de (UserAgent + résolution + langue), comparée aux autres présences de la séance.	un téléphone pour deux comptes
4	Comparaison à temps constant — <code>crypto.timingSafeEqual</code> , durée indépendante du préfixe correct.	timing attack

Tableau 6 — Les quatre couches

8.1 Scénarios concrets et réponses du système

#	Attaque	Défense
1	Photo du QR transmise à un absent L'étudiant prend une photo de l'écran et l'envoie à un ami resté chez lui.	Le QR change toutes les 10 s — la photo est expirée avant que le message n'arrive (C2).
2	Scan en visioconférence depuis l'extérieur L'ami absent ouvre un appel vidéo et scanne l'écran via la caméra distante.	Le GPS du téléphone scanneur est à plusieurs centaines de mètres → $v = 0$ (C3).
3	Un seul téléphone, deux comptes L'étudiant scanne avec son compte, se déconnecte, se reconnecte avec le compte d'un ami, rescanne.	Empreinte identique détectée → les deux présences passent en non vérifiée (C5).
4	Bypass direct de l'API (sans caméra) L'étudiant ouvre la console du navigateur et envoie un POST avec le <code>studentId</code> d'un absent.	Le serveur ignore le <code>studentId</code> du body et lit l'identité depuis le JWT signé.
5	Brute-force du token via le réseau Un script essaie des millions de tokens hex aléatoires.	$2^{32} \approx 4,3$ milliards de possibilités × fenêtre de 10 s → impossible dans le délai (C4).
6	Manipulation de l'horloge du téléphone L'étudiant change l'horloge pour faire valider un vieux token capturé hier.	La fenêtre est calculée côté serveur ; tolérance ± 60 s seulement.

Tableau 7 — Six scénarios de fraude testés

§ 9 — Stack technique

Tous les composants sont open-source. Le déploiement et la base de données sont sur des offres gratuites : coût d'exploitation nul. Chaque outil est choisi pour une raison précise — pas par effet de mode.

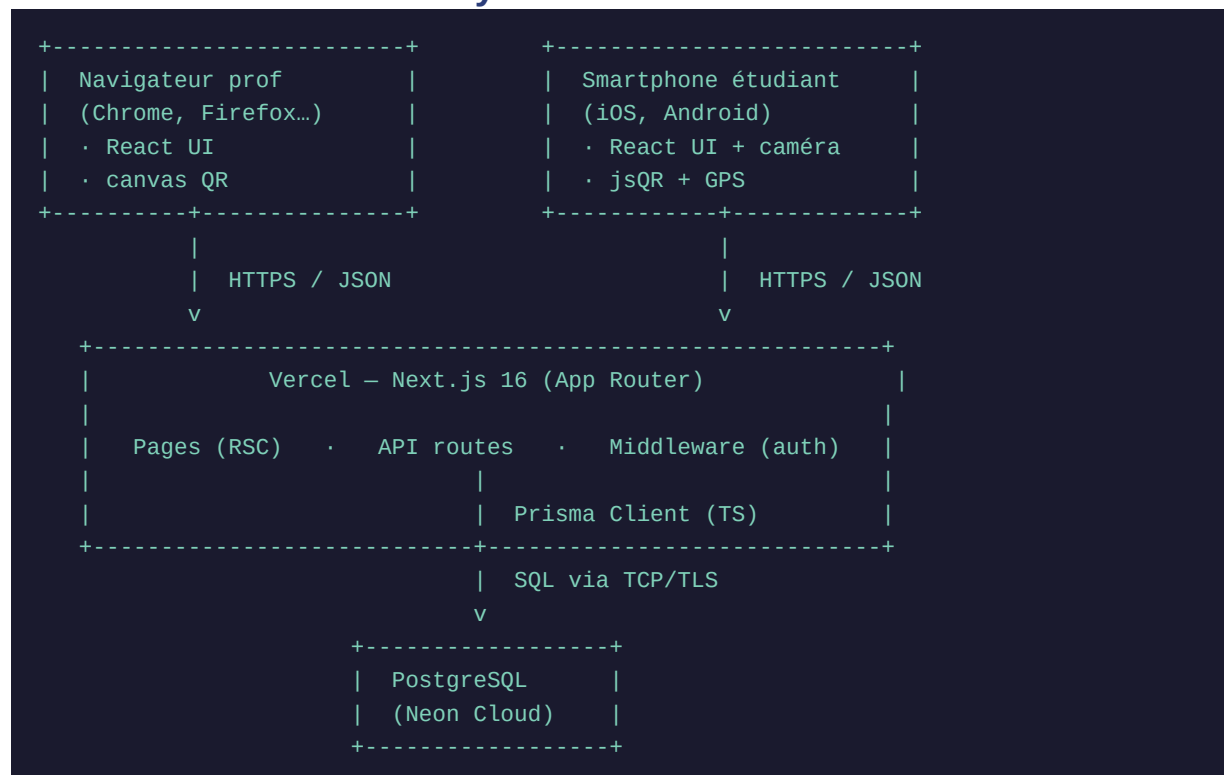
Composant	Outil	Pourquoi ce choix
Framework	Next.js 16 (App Router)	Réunit frontend et backend dans un seul projet. Déploiement Vercel sans configuration.
UI	React 19	Bibliothèque standard pour interfaces à base de composants. Mise à jour ciblée de l'écran sans rechargement.
Langage	TypeScript	Typage statique : les erreurs de structure sont attrapées à l'écriture, pas en production.
Base de données	PostgreSQL (Neon)	SGBD relationnel ACID, hébergé serverless. Supporte UNIQUE et index B-tree pour lookups $O(\log n)$.
ORM	Prisma 6	Schéma déclaratif unique, migrations versionnées, requêtes type-safe : élimine les SQL injection au niveau code.
Auth	NextAuth + bcrypt	Sessions JWT signées (24 h), mots de passe bcrypt (coût 10), comparaison à temps constant.
Validation	Zod	Schémas runtime de chaque endpoint. Rejet 400 avant toute requête SQL.
QR / Scanner	qrcode + jsQR	Génération canvas côté prof, décodage temps réel via la caméra. Tout dans le navigateur.
UI / Styles	Tailwind v4 + Radix UI	Design system cohérent, primitives accessibles (a11y), responsive desktop / mobile.
Déploiement	Vercel + GitHub	Build et déploiement automatiques à chaque push. HTTPS automatique (Let's Encrypt).

Tableau 8 — Choix techniques justifiés

§ 10 — Architecture et flux de données

Le système suit l'architecture classique **client / serveur**. Le navigateur affiche l'interface (le client) et envoie des requêtes au serveur Next.js, qui interroge la base PostgreSQL hébergée chez Neon. Tout passe par HTTPS.

10.1 Vue d'ensemble du système



10.2 Anatomie d'une requête HTTP type

1. L'étudiant scanne. Son navigateur compose un `POST /api/sessions/{id}/attendance` avec un corps JSON.
2. La requête arrive à Vercel via HTTPS. Vercel sert la route Next.js correspondante.
3. Le **middleware** NextAuth vérifie le cookie de session signé. Si invalide ou expiré → 401.
4. Le code de la route appelle `requireApiRole(["STUDENT"])` qui lit le JWT et vérifie le rôle.
5. Zod valide le corps : `token` string, `timestamp` entier, `lat/lon` optionnels.
6. Prisma exécute des `SELECT/UPDATE` en SQL, sur Neon, via TLS.
7. La réponse JSON est renvoyée — le navigateur la lit et met à jour l'interface.

DÉFINITION — JWT (JSON Web Token)

Format standard de token d'authentification. Un JWT contient trois parties séparées par des points : `en-tête.payload.signature`. Le serveur signe avec un secret ; le navigateur le renvoie à chaque requête dans un cookie. Le serveur vérifie la signature avant de faire confiance au contenu.

§ 11 — Modèle de données

Le schéma est conçu pour refléter l'organisation pédagogique réelle de l'EIDIA. Dix entités principales, reliées par des clés étrangères ; tout est normalisé (pas de duplication d'information). Le fichier source est `prisma/schema.prisma`.

Entité	Rôle	Champs clés
User	Compte utilisateur générique (admin, prof, étudiant).	id, email unique, passwordHash, role, firstName, lastName
Department	Faculté ou école (EIDIA).	name, code unique
Program	Filière d'études (GI, IASD).	name, code unique, departmentId
Group	Classe physique (GI-S5-A, GI-S5-B...).	name, programId, semester
Professor	Profil enseignant (1-1 avec User).	userId, employeeId, departmentId
Student	Profil étudiant (1-1 avec User).	userId, studentId unique, enrollmentYear
Room	Salle de cours géolocalisée.	latitude, longitude, radius, capacity
Course	Cours d'un semestre, par filière, par prof.	code unique, professorId, programId
Session	Séance planifiée d'un cours.	courseId, roomId, date, status, qrSecret, qrRotationSec
Attendance	L'enregistrement de présence d'un étudiant à une séance.	UNIQUE(sessionId, studentId), status, scannedAt, verified, lat/lon, deviceHash

Tableau 9 — Les dix entités du modèle

11.1 Les relations

```
Department --1:N-> Program --1:N-> Group --N:N-> Student
      |
      +---1:N-> Course --N:N-> Group |
            |
            +---1:N-> Session |
                  |
                  +--1:N-> Attendance
```

Une seconde lecture utile : les **contraintes d'intégrité** (clés étrangères + UNIQUE) sont déclarées une fois pour toutes dans le schéma. Toute tentative d'écriture violant l'intégrité est rejetée par PostgreSQL *avant* d'atteindre la moindre ligne de code applicatif.

§ 12 — Sécurité API par rôle

Chaque endpoint API exige un rôle minimum et, quand pertinent, une vérification de propriété (le prof ne peut agir que sur ses propres séances). Le helper `requireApiRole` est appelé en première ligne de chaque route et renvoie 401 / 403 si la condition n'est pas remplie.

AUTHENTIFICATION VS. AUTORISATION

Authentification = prouver qui on est. **Autorisation** = prouver qu'on a le droit de faire telle action. Deux niveaux distincts, à vérifier indépendamment.

Endpoint	Rôle requis	Vérifications supplémentaires
POST /api/sessions	PROFESSOR, ADMIN	le prof ne peut créer que pour lui-même
POST /api/sessions/[id]/activate	PROFESSOR, ADMIN	séance doit lui appartenir
POST /api/sessions/[id]/deactivate	PROFESSOR, ADMIN	séance doit lui appartenir
GET /api/sessions/[id]/qr-token	PROFESSOR, ADMIN	séance lui appartient — empêche un étudiant de lire le token
POST /api/sessions/[id]/attendance	STUDENT	studentId lu depuis le JWT, jamais du body
PATCH /api/sessions/[id]/attendance	PROFESSOR, ADMIN	vérifie que la présence appartient à cette séance
POST/DELETE /api/departments, programs, ...	ADMIN	réservé aux administrateurs

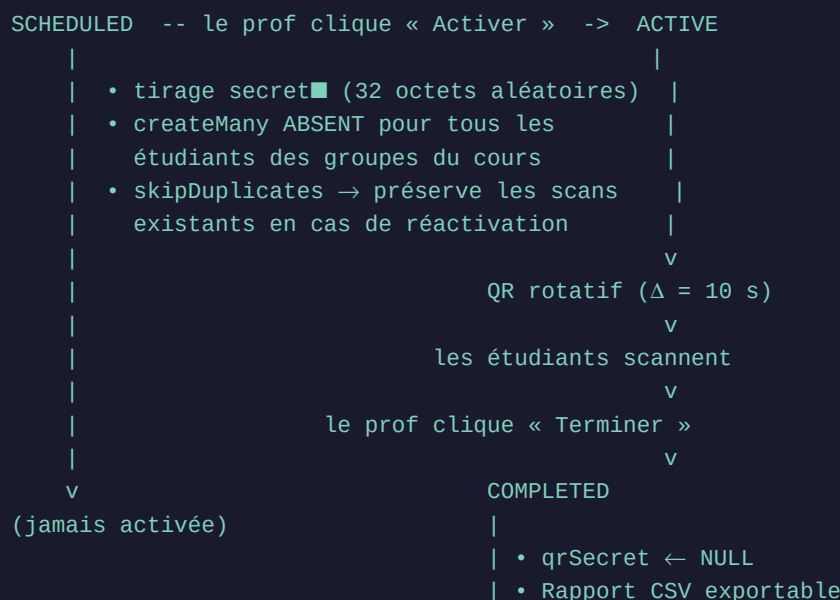
Tableau 10 — Tableau d'autorisation des endpoints

12.1 Mesures complémentaires

- **Hash bcrypt des mots de passe** (coût 10). bcrypt est volontairement coûteux à calculer.
- **Messages de login non-distinctifs**. « Email ou mot de passe incorrect » dans tous les cas.
- **bcrypt exécuté même si l'email n'existe pas** (contre un hash factice) — supprime la possibilité de distinguer les cas par *timing oracle*.
- **Validation Zod stricte**. Toute requête malformée est rejetée avec un 400 avant d'atteindre la base.
- **HTTPS partout** — fourni gratuitement par Vercel.

§ 13 — Cycle de vie d'une séance

Une séance traverse trois états : **SCHEDULED** (planifiée, pas encore commencée), **ACTIVE** (en cours, QR rotatif diffusé), **COMPLETED** (terminée, QR invalidé, rapport disponible). Les transitions sont déclenchées par le professeur.



13.1 SCHEDULED → ACTIVE (activation)

Quand le professeur clique « Activer », le serveur :

- Vérifie l'authentification et l'autorisation (rôle PROFESSOR, propriétaire de la séance).
- Génère *secret*, via `crypto.randomBytes(32)` — 256 bits d'entropie cryptographique.
- Exécute une transaction : `UPDATE session SET status='ACTIVE', qrSecret=... + INSERT INTO attendances en masse, un ABSENT par étudiant inscrit aux groupes du cours.`
- Le `skipDuplicates` garantit qu'une réactivation accidentelle ne supprime pas les présences déjà scannées.

13.2 ACTIVE → COMPLETED (terminaison)

Quand le professeur clique « Terminer » :

- Le serveur passe `status` à `COMPLETED`.
- Surtout : il met `qrSecret` à `NULL`. Tous les tokens déjà émis deviennent invalidables — il n'y a plus de référence cryptographique avec laquelle les vérifier.
- Le rapport devient disponible dans la section « Rapports » : liste filtrable, statistiques par cours, export CSV.

POURQUOI EFFACER LE SECRET ?

Le secret est *la seule donnée* qui permet de valider un token QR. Tant qu'il existe en base, un attaquant qui réussirait à le copier pourrait théoriquement valider des tokens. En l'effaçant à la fermeture, on minimise la fenêtre d'attaque : **aucun secret survivant n'est exploitable** une fois la séance close.

§ 14 — Bilan et limites assumées

Ce projet illustre comment un problème universitaire concret se laisse **entièrement formaliser dans le langage de la Recherche Opérationnelle** — variables binaires de décision, ensembles, fonction objectif multi-critère pondérée, contraintes mathématiquement explicites — puis se transforme en application web complète, déployée et utilisable.

14.1 Indicateurs clés

Indicateur	Valeur
Gain de temps par séance (Z■)	~96 %
Fenêtre de validité d'un token QR	10 s
Couches anti-fraude indépendantes	4
Étudiants EIDIA dans la base	201
Groupes (4 GI + 4 IASD)	8
Cours actifs	4 (RO, RSE, MFA, GIN)
Coût d'exploitation annuel	0 €

Tableau 11 — Indicateurs clés

14.2 Correspondance théorie ↔ implémentation

Concept RO	Implémentation technique
Variable binaire $x_{ij} \in \{0,1\}$	colonne status (ABSENT / PRESENT / LATE / EXCUSED)
Contrainte d'unicité (C1)	contrainte SQL UNIQUE(sessionId, studentId)
Contrainte temporelle (C2)	HMAC-SHA256 avec compteur de fenêtres $k = \blacksquare t / 10 \blacksquare$
Contrainte géographique (C3)	formule de Haversine + Room.radius par salle
Contrainte cryptographique (C4)	secret 32 octets ; timingSafeEqual à la vérification
Contrainte d'appareil (C5)	SHA-256 de (UserAgent + résolution + langue), dédup par séance
Fonction objectif multi-critère Z	tableau de bord prof — couverture, taux vérifié, temps
Décomposition par séance	indépendance des séances → scalabilité horizontale

Tableau 12 — De la formalisation au code

14.3 Limites assumées

- Le **GPS smartphone** a une précision de 5 à 20 m en intérieur — d'où le statut « non vérifié » plutôt qu'un rejet automatique.

- Le **fingerprint d'appareil** est heuristique, pas une preuve forte : deux modèles identiques produisent la même empreinte. Décourage la fraude opportuniste, pas un attaquant déterminé.
- L'**horloge du smartphone** est vérifiée côté serveur (tolérance ± 60 s). Au-delà, le scan est rejeté avec message explicite.
- Le **scan exige la caméra** — donc HTTPS. En réseau interne sans HTTPS, la caméra refuse l'accès dans les navigateurs modernes.

14.4 Pistes futures

- Analyse temporelle de l'assiduité par cours et par étudiant ; détection automatique des décrochages.
- Alertes au professeur quand un étudiant dépasse un seuil d'absences.
- Export PDF des feuilles de présence en complément du CSV.
- Mode hors-ligne pour les zones à mauvaise couverture réseau (synchronisation différée).
- Intégration directe avec Konosys pour synchronisation automatique des rosters.

§ 15 — Glossaire technique

15.1 Recherche Opérationnelle

Terme	Définition
Variable de décision	Inconnue que le modèle fixe. Ici, $x_{ij} \in \{0,1\}$ = présence.
Fonction objectif	Quantité scalaire à maximiser ou minimiser.
Contrainte	Condition à respecter. Inégalité, égalité, ou logique.
Optimisation combinatoire	Optimisation sur un ensemble fini de solutions discrètes.
Multi-objectif	Plusieurs critères à optimiser. Approche : scalarisation pondérée.
Analyse de sensibilité	Étude de la variation de la solution optimale en fonction des paramètres.

15.2 Cryptographie et sécurité

Terme	Définition
HMAC-SHA256	Code d'authentification de message basé sur SHA-256. Prouve intégrité + authenticité.
TOTP (RFC 6238)	Time-based One-Time Password. Schéma qui dérive un mot de passe court d'un secret partagé et du temps. Notre QR rotatif s'en inspire.
bcrypt	Fonction de hashage de mots de passe volontairement coûteuse, résiste au brute-force.
JWT	JSON Web Token : format standard de token d'auth signé par le serveur.
Timing attack	Attaque qui mesure le temps de réponse pour deviner des informations.
Défense en profondeur	Stratégie qui superpose plusieurs lignes de défense indépendantes.
Fingerprint	Empreinte numérique d'un appareil dérivée de ses caractéristiques.

15.3 Web et développement

Terme	Définition
Frontend	Partie du logiciel qui tourne dans le navigateur de l'utilisateur.
Backend	Partie qui tourne sur un serveur (API, BDD, logique métier).
API REST	Style d'API où chaque ressource a une URL et où l'on agit dessus via GET, POST, PATCH, DELETE.
SGBD relationnel	Système de gestion de base de données basé sur les tables et relations.
ACID	Atomicité, Cohérence, Isolation, Durabilité — propriétés des transactions.

Terme	Définition
ORM	Object-Relational Mapper : pont entre objets et tables SQL.
HTTPS	HTTP chiffré via TLS. Empêche l'écoute du trafic réseau.
CSV (RFC 4180)	Format texte : lignes de valeurs séparées par des virgules.

15.4 Géolocalisation et QR

Terme	Définition
Haversine	Formule qui calcule la longueur du plus court arc géodésique entre deux points sur une sphère.
Geofence	Zone géographique définie par un point central et un rayon.
QR code	Code-barres 2D capable d'encoder du texte, lu par une caméra.
Token rotatif	Token cryptographique qui change automatiquement à intervalle régulier.

Code source complet : github.com/MouhssineElBoumshouli/Student-Attendance-System

Stack : Next.js 16 · React 19 · TypeScript · Prisma 6 · PostgreSQL (Neon) · NextAuth · Tailwind v4 · déployé sur Vercel